OPERATING SYSTEMS PROJECT · OCTOBER 2026

OS Behavior Monitor

Divergence-Gated Behavioral Baseline Learning with Overhead-Budgeted Adaptive Sampling

Team: Gracy Mehndiratta (24BCE2987) and Manan Kulshrestha (24BCE0262)

Repository: github.com/gracymehndiratta/os-project

Document: Complete project overview, algorithms, and technology stack

A per-tick operating-system monitoring agent that learns per-(application, context) behavioral baselines, detects anomalies with a joint multi-resource Mahalanobis score, protects those baselines from slow poisoning through a divergence gate with quarantine and independent corroboration, and keeps its own CPU overhead inside a target budget using adaptive tiered sampling.

1. What the project is

OS Behavior Monitor is an operating-system monitoring agent written in Python. It learns what "normal" behavior looks like for every application on a machine, then raises explainable alerts when a running process deviates from that learned behavior.

The project solves two problems together:

1. **Slow-poisoning attacks.** An attacker gradually changes a process so the monitor "learns" the malicious behavior and stops alerting. This is the "boiling frog" or slow-ramp attack.
2. **Monitor overhead.** A monitor that samples everything all the time becomes a resource hog itself.

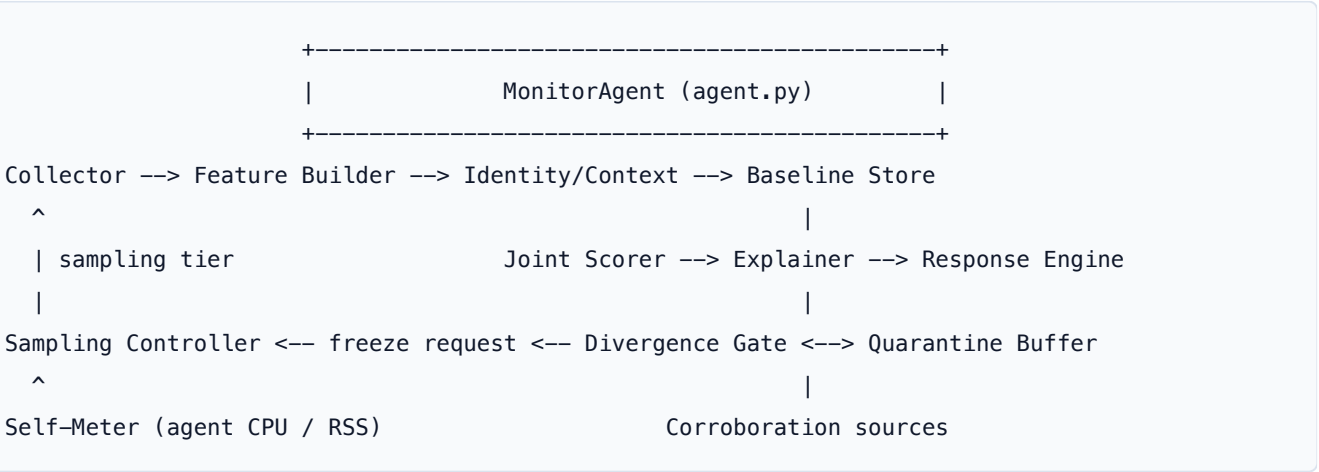
The solution is a per-tick pipeline with **two feedback loops plus an interlock**:

- **Loop 1 - divergence gate (poisoning control).** Compares short-term, long-term, and trusted-reference baselines; freezes the long-term baseline on suspicious drift; quarantines new samples; only merges them after independent corroboration.
- **Loop 2 - sampling controller (overhead control).** Measures the agent's own CPU and RSS and adjusts a per-tick cost budget to stay near a CPU target, assigning sampling tiers in priority order.
- **Interlock.** A freeze flags the process and raises its sampling priority; a `THROTTLE` response also escalates its tier.

Scope note: all attack traces are synthetic. No claim is made about detection performance on real malware. Kernel-level telemetry (eBPF/ETW), fleets, and cloud features are explicitly out of scope.

---

2. High-level architecture



Per-tick data flow

1. **Sampling controller** assigns a tier to every process, in priority order, within the budget.
2. **Collector** returns samples for processes due at that tier. Tier 0 only returns CPU and memory.
3. Each full sample becomes a **log-scale feature vector**, keyed by application identity and context.
4. **Baseline store** picks the context-specific baseline if trusted; otherwise the coarse `*` baseline.
5. **Scorer** computes the joint deviation `D^2` and a chi-squared risk value.

6. **Divergence gate** updates drift measures and decides to learn, freeze, merge, or discard.
7. **Alerts** fire after several consecutive high-risk samples, carrying ranked feature contributions.
8. **Self-meter** reads the agent's own usage; the controller adjusts the budget.

During warm-up (`baseline.min_samples = 30`) every sample is used for learning and nothing is scored.

---

3. Algorithms used

3.1 Feature engineering

Five canonical features per process:

#	Feature	Unit	Notes
0	`cpu_pct`	% CPU	CPU utilization
1	`rss_mb`	MB	Resident memory
2	`disk_io_kbps`	KB/s	Combined read + write
3	`net_kbps`	KB/s	Live mode uses open-socket count as a proxy
4	`child_rate`	procs/s	Rate of new child processes

Each raw vector is transformed with:

```
x' = log1p(max(x, 0))
```

The log transform makes heavy-tailed resources more additive and makes the Gaussian assumption of the scorer more reasonable.

3.2 Stable identity and context keys

- **Application identity:** SHA-256 of the normalized executable name, truncated to 16 hex characters. It survives PID reuse.
- **Process instance identity:** `pid@start_time`.
- **Context:** `time-bucket, username, parent`, where time bucket is `night`, `morning`, `afternoon`, or `evening` in UTC.
- **Baseline key:** `(application_id, context)`; the coarse fallback context is `*`.

This prevents, for example, a midnight cron job from being scored against a daytime interactive baseline.

3.3 Incremental Gaussian baselines

`Gaussian` stores an effective count `n`, a mean vector, and a covariance matrix. It supports three operations.

Exact Welford online update for reference and merged summaries:

```
n'   = n + 1
delta = x − mu
mu'  = mu + delta / n'
Sigma' = (n*Sigma + delta*(x − mu')^T) / n'
```

Exponentially weighted update for short and long baselines:

```
mu'    = mu + alpha*(x − mu)
Sigma' = (1 − alpha)*(Sigma + alpha*delta*delta^T)
```

with `alpha_short = 0.1` and `alpha_long = 0.01`.

Exact parallel merge of two Gaussians:

```
n  = n1 + n2
mu = mu1 + (mu2 − mu1)*n2/n
Sigma = [n1*Sigma1 + n2*Sigma2 + (n1*n2/n)*(mu2−mu1)(mu2−mu1)^T] / n
```

The merge is used when quarantined samples are reconciled after corroboration.

The store keeps up to **256** `(application, context)` keys with **LRU eviction**, and creates and keeps a coarse `*` fallback per application.

3.4 Joint Mahalanobis anomaly scoring

Against the long-term baseline:

```
delta = x − mu
D^2   = delta^T (Sigma + lambda*I)^−1 delta
risk  = chi2.cdf(D^2, df = 5)
```

- `lambda = 0.02` is the ridge regularization.
- The covariance captures **correlations** between resources, so combinations of moderate deviations are detected and correlated features are not double-counted.
- Risk is converted to a percentile-like chi-squared CDF value.

Level	Risk threshold	Meaning
`NOTIFY`	>= 0.995	Notify administrator
`THROTTLE`	>= 0.9995	Escalate sampling and notify
`SUSPEND`	>= 0.99995	Consider suspension

3.5 Explainability and feature attribution

The contribution decomposition is exact:

```
c_i = delta_i * (Sigma^-1 delta)_i / D^2
```

Contributions are sorted by absolute magnitude, and each gets a z-score `delta_i / sigma_i` and a direction (above or below baseline). Alerts list the top features in plain language, for example:

```
cpu_pct: 62.3% of deviation (above baseline, z=4.1)
```

3.6 Divergence gate: three drift detectors

The gate protects the long baseline from slow poisoning.

a) Mean shift, short versus long:

```
shift = sqrt( (mu_short - mu_long)^T Sigma_ref^-1 (mu_short - mu_long) / d )
```

Freeze when `shift >= 1.5`.

b) CUSUM, accumulated slow excess:

```
e_t = ( (x - mu_long)^T Sigma_ref^-1 (x - mu_long) - d ) / sqrt(2*d)
S_t = max(0, S_{t-1} + (e_t - k))
```

with slack `k = 0.5` and alarm `h = 12.0`. Freeze when `S_t >= 12`.

c) Reference drift, long versus trusted reference snapshot:

```
ref = sqrt( (mu_long - mu_ref)^T Sigma_ref^-1 (mu_long - mu_ref) / d )
```

Freeze when `ref >= 1.0`.

Key detail: drift measures are standardized by the **trusted reference covariance**, not the live long covariance, so an attacker cannot hide a ramp by inflating the long baseline's variance. The reference snapshot is taken once the long baseline has converged, at `2 * min_samples = 60` observations.

3.7 Gate state machine

```
LEARNING --(min_samples)--> NORMAL --(mean shift | CUSUM | reference drift)--> FROZEN
                              ^                                                  |
                              |       independent corroboration --> MERGE        |
                              |       TTL expired ----------------> DISCARD      |
                              +--------------------------------------------------+
```

- **LEARNING:** warm-up, nothing is scored.
- **NORMAL:** short baseline always learns; long baseline learns only if the sample's distance is below `update_guard = 3.0`.
- **FROZEN:** long baseline locked; samples go into quarantine; process flagged.

- **MERGE:** buffer summary merged into the long baseline after independent corroboration; reference reset; CUSUM cleared.
- **DISCARD:** buffer dropped after TTL expiry; short baseline reset to long; the gate may freeze again if behavior continues.

Corroboration is strict. Only an administrator confirmation or a verified application update counts. A return to normal behavior is never treated as corroboration, because malware can stop on purpose.

3.8 Overhead-budgeted adaptive sampling

Tiers:

Tier	Sampling interval	Features	Cost per tick
0 - MINIMAL	every 8 ticks	CPU + memory	0.125 units
1 - NORMAL	every 2 ticks	all five	2.0 units
2 - DETAILED	every tick	all five	4.0 units

Tier assignment (greedy, budget-constrained):

1. Every process is guaranteed tier 0; base cost is `n * 0.125`.
2. Processes are sorted: flagged first, then highest desired tier, then stable by key.
3. Each process gets the highest tier it can afford.
4. Any tick where a process gets less than desired counts as **reduced coverage**.

Budget adaptation with hysteresis:

```
if cpu > target:            budget <- budget * 0.80; violations += 1
elif cpu <= 0.70 * target:  dwell += 1
                            if dwell >= 3: budget <- budget * 1.15; dwell <- 0
else:                       dwell <- 0
budget <- clamp(budget, 4, 512)
```

The default target is **1% of one core**, measured over a **20-tick** sliding window. The asymmetric up and down thresholds and dwell count prevent oscillation.

3.9 Graduated response with dry-run and rollback

Risk level	Severity	Response
NORMAL	none	log only
NOTIFY	medium	notify administrator
THROTTLE	high	raise sampling tier and notify
SUSPEND	critical	consider suspension (dry-run unless enabled)

- Alerts fire after **3 consecutive** high-risk samples.
- THROTTLE can raise the process to DETAILED tier through an `on_throttle` callback.

- SUSPEND records a reversible `RollbackAction`; if not dry-run and `auto_suspend` is on, it sends `SIGSTOP`, and rollback sends `SIGCONT`.
- **Dry-run is the default.**

3.10 Isolation-forest ablation (not the core algorithm)

`agent/isolation.py` implements a conventional **Isolation Forest** detector as an ablation for comparison with prior work. It is not part of the proposed solution:

- One **global** forest with `n_estimators = 100`, refit every 120 ticks on a 256-sample sliding window.
- First fit after 128 samples.
- Raw score is negated so higher means more anomalous.
- Converted to risk via an **empirical tail probability**:

```
rank = number of r in window with r <= raw
risk = (rank + 0.5) / (n + 1)
```

- Explanations use **robust departure from median and IQR** (`IQR / 1.349` as sigma), because iForest has no closed-form per-feature attribution.
- No gate and no budget; it is a pure point-anomaly baseline.

It is deliberately included to show that a point-anomaly detector misses slow collective drift, is context-blind, and costs roughly an order of magnitude more per sample.

3.11 Latency tracking

A `LatencyTracker` keeps a rolling window of per-sample processing times and reports **p50, p95, and p99** in milliseconds, exposed in agent status.

---

4. Module map

Module	Responsibility
`backend/agent/collector.py`	`ReplayCollector` (synthetic traces) and `PsutilCollector` (live sampling)
`backend/agent/features.py`	Log-scale features, stable application identity, time and parent context
`backend/agent/baseline.py`	`Gaussian`, `QuarantineBuffer`, `KeyState`, `BaselineStore`
`backend/agent/scoring.py`	Regularized Mahalanobis `D^2` and ranked contributions
`backend/agent/gate.py`	Mean shift, CUSUM, reference drift, state machine, quarantine, corroboration
`backend/agent/sampling.py`	`SelfMeter`, `SamplingController`, `LatencyTracker`
`backend/agent/response.py`	Graduated responses, dry-run, rollback
`backend/agent/agent.py`	Orchestrates the pipeline and both loops
`backend/agent/config.py`	All tunables in one place

Module	Responsibility
`backend/agent/data_model.py`	`Sample`, `BaselineKey`, `Tier`, `GateState`, `RiskLevel`, `Alert`
`backend/agent/isolation.py`	Isolation-forest ablation
`backend/agent/scenarios.py`	Synthetic traces for the experiments and live demo
`backend/agent/evaluation.py`	Variants, scenario runner, metrics table
`backend/agent/api.py`	FastAPI REST `/api` and WebSocket `/ws`
`backend/agent/models.py`	Pydantic response models
`backend/simulator.py`	Standalone demo, replay, and live runner
`backend/evaluate.py`	Evaluation harness CLI

5. Configuration defaults

All parameters live in `backend/agent/config.py`.

Parameter	Default	Meaning
`baseline.alpha_short` / `alpha_long`	0.1 / 0.01	EWMA rates
`baseline.min_samples`	30	Samples before a baseline is trusted
`baseline.ridge`	0.02	Covariance regularization
`baseline.max_entries`	256	LRU cap on baseline keys
`gate.shift_thr`	1.5	Mean-shift freeze threshold
`gate.cusum_k` / `cusum_h`	0.5 / 12.0	CUSUM slack and alarm
`gate.ref_thr`	1.0	Reference-drift freeze threshold
`gate.update_guard`	3.0	Distance above which the long baseline skips learning
`gate.ttl_days`	1.0	Quarantine lifetime, 1440 ticks
`gate.max_buffers` / `max_buffer_samples`	64 / 512	Buffer caps
`score.notify` / `throttle` / `suspend`	0.995 / 0.9995 / 0.99995	Chi-squared risk thresholds
`score.alert_consecutive`	3	Consecutive high-risk samples for an alert
`sampling.cpu_budget_pct`	1.0	Target agent CPU percentage
`sampling.window_ticks`	20	Self-meter window
`sampling.budget_shrink` / `budget_grow`	0.20 / 0.15	Budget change per window

Parameter	Default	Meaning
`sampling.grow_threshold` / `dwell_count`	0.70 / 3	Growth hysteresis
`sampling.initial/min/max_budget`	64 / 4 / 512	Cost-budget bounds
`dry_run`	True	Disruptive responses logged, not executed
`ticks_per_day`	1440	Replay ticks per simulated day

Two values were tuned from design defaults after experiments: diurnal amplitude `0.08` and gate ridge `1e-3`.

---

6. Tech stack

Backend

Technology	Version	Role
Python	3.12	Agent, statistics, gate, sampling, evaluation, API
FastAPI	>= 0.111	REST and WebSocket API
Uvicorn	>= 0.29	ASGI server
Pydantic	>= 2.7	API response models
NumPy	>= 1.26	Vectors, covariance, linear algebra
SciPy	>= 1.13	Chi-squared CDF
psutil	>= 5.9.8	Live OS process telemetry
websockets	>= 12.0	WebSocket transport
scikit-learn	>= 1.5	Isolation-forest ablation only, lazy import
python-multipart	>= 0.0.9	FastAPI dependency

Frontend

Technology	Version	Role
React	18.3	UI components
TypeScript	5.5	Typed application logic
Vite	5.3	Dev server and bundler; proxies `/api` and `/ws` to port 8000
Recharts	2.12	Charting library dependency
TailwindCSS	3.4	Styling
PostCSS / Autoprefixer	current	CSS pipeline

Styling

A custom **dark glassmorphism security-dashboard** design system lives in `frontend/src/index.css`: slate background, blurred translucent cards, colored severity badges, a glowing FROZEN state, skeleton loaders, and slide-in and pulse animations.

Tooling

- **Python unittest** for tests, with no pytest dependency.
- **Node.js 23 / npm 10** for the frontend build.
- **Git and GitHub** for version control.
- Developed on **macOS (Darwin)** with Python 3.12.4; the backend is portable to Linux.

---

7. REST and WebSocket API

Endpoint	Method	Description
`/`	GET	Service info
`/api/status`	GET	Agent status: tick, tier, CPU and RSS, budget, violations, latency
`/api/alerts`	GET	List alerts with filters `acknowledged`, `severity`, `limit`
`/api/alerts/{id}/acknowledge`	POST	Acknowledge an alert
`/api/baselines`	GET	All active baselines
`/api/baselines/pid/{pid}`	GET	Baselines for one process
`/api/baselines/pid/{pid}/reconcile`	POST	Merge quarantine after corroboration
`/api/baselines/pid/{pid}/dismiss`	POST	Discard quarantine
`/api/processes`	GET	Monitored processes with risk and tier
`/api/processes/{pid}/tier`	POST	Manually set sampling tier
`/api/budget`	GET	Budget controller status
`/api/rollback-actions`	GET	Recorded reversible actions
`/api/rollback/{action_id}`	POST	Roll back an action
`/ws`	WebSocket	Pushes status every 2 seconds

FastAPI auto-documentation is served at `/docs`.

---

8. Dashboard

The React dashboard has **four tabs**:

1. **Overview** - status cards for processes, alerts, frozen baselines, and budget; radial gauges for CPU, memory, and I/O; a live overhead chart; and recent alerts.
2. **Alerts** - explainable alert feed with severity badges, a plain-language explanation, top-three contributing features with bars and z-scores, and an acknowledge button.
3. **Baselines** - short-term versus long-term statistics per feature, gate state, quarantine count, CUSUM, freeze count, and a Reconcile button for frozen keys.
4. **Processes** - sortable table of PID, name, CPU, memory, risk level and score, baseline state, sampling tier, and parent process.

Data flow: WebSocket for live status, REST polling every 4 seconds for alerts, baselines, and processes.

---

9. Simulation and attack scenarios

The replay collector ships with benign processes: **chrome, vscode, python3, node, postgres, and nginx**.

The **demo scenario** adds four synthetic attack patterns:

Process	Attack	Behavior
`malware_sim`	`ramp`	Slow multi-resource ramp, the boiling-frog pattern
`exploit_py`	`spike`	Sudden CPU and I/O burst
`leaky_app`	`leak`	Continuous memory growth
`bash_fork`	`fork`	Rapid child-process spawning, a fork bomb

Traces include a diurnal sine component, per-process noise, and a deterministic seeded RNG.

---

10. Evaluation harness

Variants and ablations

Variant	Gate	Budget	Learning	Detector
`static`	off	off	static, never adapts	Gaussian
`plain_adaptive`	off	off	adaptive	Gaussian
`gate_only`	on	off	adaptive	Gaussian
`budget_only`	off	on	adaptive	Gaussian
`full`	on	on	adaptive	Gaussian
`iforest`	off	off	adaptive	Isolation Forest

Scenarios

Scenario	Simulates	Measures
`benign`	Normal diurnal behavior	False positives per process-day
`slow_ramp`	Gradual multi-resource ramp over days	Detection rate and delay
`evading_ramp`	Ramp slow enough to hide from the short-versus-long gate	Whether reference drift catches it
`legit_step`	Permanent benign step plus admin confirmation	Alerts after the step and recovery time
`overhead`	60 to 200 processes under a CPU budget	CPU, violations, reduced coverage

Each variant gets a **freshly rebuilt scenario**, so all variants see the same noise stream.

Reported metrics

- Detection rate and delay in simulated days.
- False-positive episodes per process per day.
- Alerts after a legitimate step and recovery time.
- Agent CPU, budget violations, and reduced-coverage ticks.
- p50, p95, and p99 per-sample latency.

Key reference results from the quick run

- **Benign:** 0 false-positive alerts for every variant, and 0 benign freezes for gate variants.
- **Slow ramp:** `plain_adaptive` produces only 1 late alert at 1.17 days; `gate_only` and `full` produce **433 sustained alerts** and detect at **0.92 days**.
- **Evading ramp:** `plain_adaptive` misses it entirely; gate variants detect it at **1.92 days** through **reference drift**, with 466 alerts.
- **Legit step:** `plain_adaptive` recovers fast, with 4 alerts after 0.25 days, but is poisonable; gate variants freeze and recover only after admin corroboration, about 0.50 days and 480 alerts. This is the honest tradeoff.
- **Overhead:** the budget controller reduces coverage for 15 ticks and keeps modeled CPU near 0.23 percent, with no budget violations.
- **Isolation forest:** misses both ramps, raises no alerts on `legit_step`, and produces **0.64 false positives per process per day** on the heterogeneous overhead fleet. It costs roughly an order of magnitude more per sample.
- **Latency:** the Gaussian scorer is about 0.18 ms p50 and 0.37 ms p95, machine dependent; iForest is about 4 to 6 ms p50.

---

11. Tests

There are **30 tests** in two files, run with the standard-library `unittest` runner:

```
cd backend
python -m unittest discover -s tests
```

test_collector_sampling_response.py, 9 tests

- Replay collection at the default tier.
- Tier intervals and feature masks.
- Live psutil collection.
- Self-meter mock measurement.
- Greedy tier assignment and flagged priority.
- Budget shrink and grow.
- Consecutive scoring and alert firing.
- Dry-run and rollback action.
- Non-dry-run throttle callback.

test_pipeline.py, 21 tests

- Exact Gaussian merge matches batch statistics.
- EWMA updates.
- Context fallback to the coarse baseline.
- LRU eviction.
- Thread-safe snapshot during mutation.
- Contribution ranking and sum, and monotonic risk levels.
- Gate freeze on mean shift and quarantine.
- Corroboration only when independent.
- TTL discard.
- Freeze raises the desired tier, the interlock.
- Latency percentiles.
- Benign scenario has no false positives.
- Gate catches a slow ramp while plain adaptive does not.
- Status shape matches the dashboard.
- Isolation-forest behavior: no score before fit, outlier scoring, single-axis blind spot, and ranking.
- Agent runs with the iForest detector.
- Evaluation harness rebuilds scenarios fairly.

---

12. Project layout

```
os-project/
├── README.md
├── .gitignore
├── backend/
│   ├── main.py               # uvicorn entry point
│   ├── simulator.py          # standalone replay and live runner
│   ├── evaluate.py           # evaluation CLI
│   ├── requirements.txt
│   ├── agent/                # monitoring agent modules
│   └── tests/                # unittest suite
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.js
│   └── src/
│       ├── App.tsx
│       ├── main.tsx
│       ├── index.css
│       ├── types.ts
│       └── components/
│           ├── StatusCards.tsx
│           ├── BudgetPanel.tsx
│           ├── MetricsChart.tsx
│           ├── AlertsPanel.tsx
│           ├── BaselinesPanel.tsx
│           └── ProcessesPanel.tsx
└── docs/
    ├── REVIEW_DOCUMENT.md
    ├── OS_Behavior_Monitor_Report.docx and .pdf
    ├── report.html
    ├── screenshots/
    ├── terminal_eval.html and terminal_tests.html
    └── build scripts and evaluation outputs
```

13. How to run

Backend

```
cd backend
pip install -r requirements.txt

# API and dashboard backend, simulation mode by default
python main.py                  # http://localhost:8000, docs at /docs
# or: uvicorn agent.api:app --port 8000

# Live psutil collection
OSMON_LIVE=1 python main.py

# Faster simulated demo
OSMON_TICK_INTERVAL=0.15 python main.py

# Standalone simulator
python simulator.py --scenario demo --ticks 600
python simulator.py --scenario slow_ramp --variant full --ticks 3000
python simulator.py --live --ticks 60

# Evaluation harness
python evaluate.py --quick
python evaluate.py --scenarios slow_ramp evading_ramp --variants full plain_adaptive
```

Frontend

```
cd frontend
npm install
npm run dev       # http://localhost:3000, proxies /api and /ws to :8000
npm run build     # production bundle into dist/
```

---

14. Design decisions worth knowing

1. **Divergence gating with quarantine.** Drift freezes the long baseline; samples wait in a bounded buffer and merge only after independent corroboration.
2. **CUSUM for slow poisoning.** It catches persistent small changes that stay below an instantaneous threshold.
3. **Reference drift.** A trusted snapshot catches an attacker who moves both short and long baselines together.
4. **Interlock.** A freeze or THROTTLE raises the process sampling tier, subject to budget.

5. **Joint multi-resource scoring.** Mahalanobis with per-feature variance avoids double-counting correlated features and detects combinations of moderate deviations.
6. **Explainable alerts.** Ranked per-feature contributions with z-scores and direction.
7. **Graduated response.** Logging, notification, sampling escalation, and suspension, all recorded for rollback, with dry-run by default.

---

15. Known limitations stated by the project

- Attack traces are **synthetic**; there is no real-attack performance claim.
- Gating **reduces but does not eliminate** slow-poisoning risk.
- Reference drift also freezes on legitimate slow change, which then needs administrator confirmation.
- Requiring independent corroboration means many buffers are discarded, so baselines can go stale and raise false positives after real workload changes.
- Load can push the monitor into degraded sampling; reduced coverage is counted as a risk signal.
- A Python prototype **overstates overhead** because of interpreter cost.
- Automated suspension can disrupt legitimate processes; dry-run is the default.

---

16. Code-level notes and quirks

These are implementation observations, not part of the documented design.

- `Config.max_buffers` and `Config.warm_up_ticks` are declared but not enforced or used anywhere.
- `KeyState.trusted` checks `long.n >= 0`, which is always true for an initialized state. The meaningful part is `gate_state != LEARNING`.
- The quarantine TTL is measured from `freeze_tick`, not from the oldest buffered sample.
- `risk_score` in alerts and process views is actually the **Mahalanobis distance**, the square root of `D^2`, while `risk_level` comes from the chi-squared percentile.
- Live network is an **open-socket count**, not bytes.
- The current working tree contains **uncommitted changes** that add the isolation-forest ablation and some tuning; the README and report already describe it.
- `frontend/dist/` and `node_modules/` exist locally but are gitignored.

---

17. One-sentence summary

This is an explainable, self-throttling OS process-anomaly detector that learns per-application behavioral baselines, uses Mahalanobis scoring plus a CUSUM and reference-drift gate to resist slow poisoning, and uses a budgeted tiered sampling controller to keep its own overhead near 1 percent CPU, wrapped in a FastAPI and React dashboard and validated with a synthetic ablation study.